

# EXPERIMENT – 3

**Name:** K ANSHU

**Roll No:** 2520030440

**Section:** 8

**Title**

**Process Creation and Process State Observation using fork()**

**Aim**

To create a child process using fork(), display the **PID, PPID, and process state** of the parent and child processes, and observe the process state transitions.

**Theory**

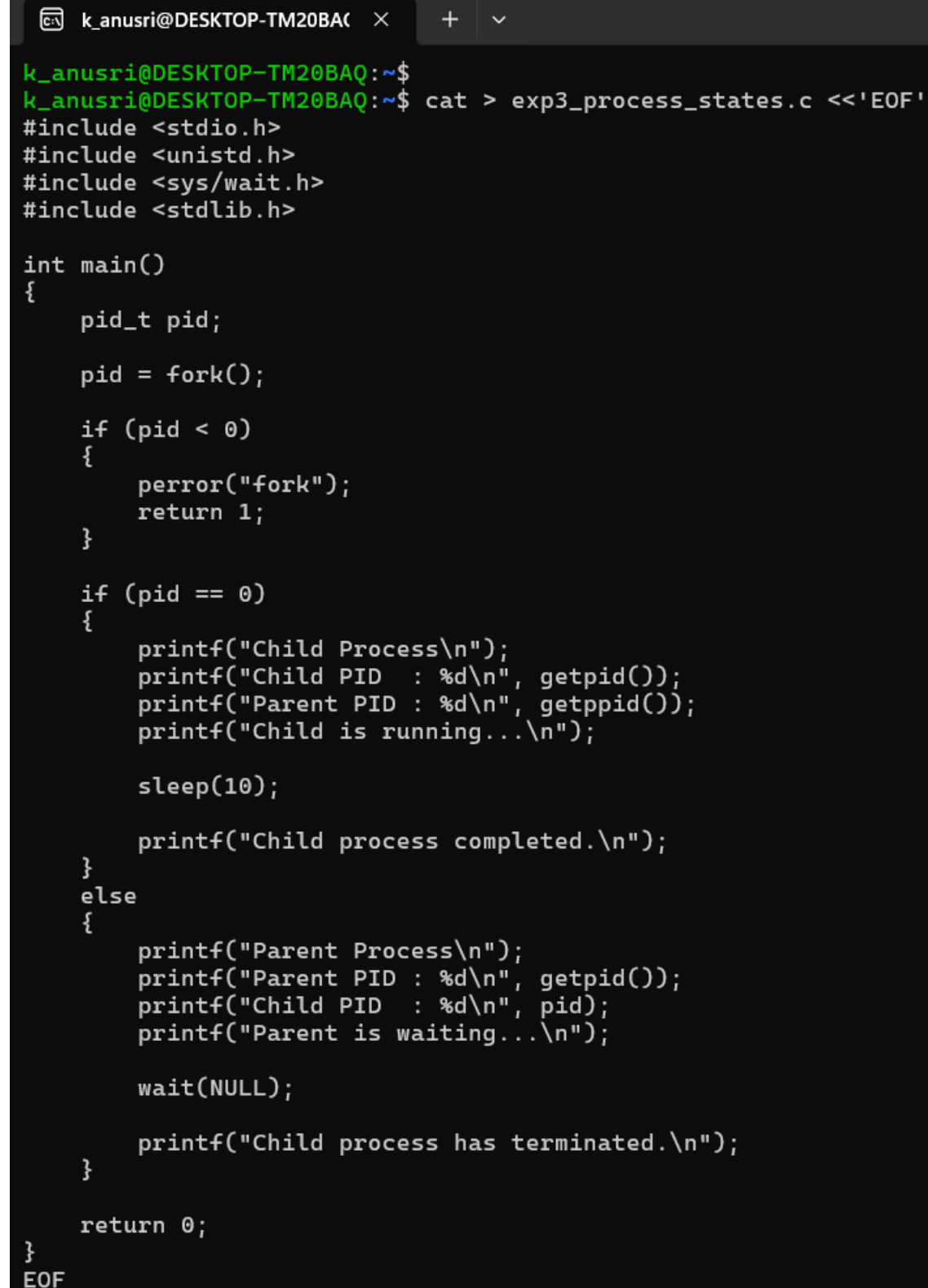
A **process** is a program in execution.

- fork() creates a new child process.
- getpid() returns the PID of the current process.
- getppid() returns the PID of the parent process.
- wait() allows the parent process to wait for the child process to complete.
- The process state can be observed using the ps command.
- Common process states include running and sleeping.

**Algorithm**

1. Start the program.
2. Call fork() to create a child process.
3. Check whether fork() was successful.
4. If fork() returns 0, execute the child-process code.
5. Display the child PID and parent PID.
6. Make the child process sleep for a few seconds.
7. In the parent process, display the parent PID and child PID.
8. Use wait() so the parent waits for the child.
9. Observe the process using the ps command.
10. Display the completion message and terminate the program.

## Program Code



```
k_anusri@DESKTOP-TM20BAQ: ~$  
k_anusri@DESKTOP-TM20BAQ: ~$ cat > exp3_process_states.c <<'EOF'  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
  
int main()  
{  
    pid_t pid;  
  
    pid = fork();  
  
    if (pid < 0)  
    {  
        perror("fork");  
        return 1;  
    }  
  
    if (pid == 0)  
    {  
        printf("Child Process\n");  
        printf("Child PID : %d\n", getpid());  
        printf("Parent PID : %d\n", getppid());  
        printf("Child is running...\n");  
  
        sleep(10);  
  
        printf("Child process completed.\n");  
    }  
    else  
    {  
        printf("Parent Process\n");  
        printf("Parent PID : %d\n", getpid());  
        printf("Child PID : %d\n", pid);  
        printf("Parent is waiting...\n");  
  
        wait(NULL);  
  
        printf("Child process has terminated.\n");  
    }  
  
    return 0;  
}  
EOF
```

## Execution and Screenshots

### Screenshot 1 — Compilation and Program Execution

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc exp3_process_states.c -o exp3_process_states
k_anusri@DESKTOP-TM20BAQ:~$ ./exp3_process_states
Parent Process
Parent PID : 497
Child PID : 498
Parent is waiting...
Child Process
Child PID : 498
Parent PID : 497
Child is running...
Child process completed.
Child process has terminated.
```

### Screenshot 2 — Process State Observation

```
k_anusri@DESKTOP-TM20BAQ:~$ ./exp3_process_states &
[1] 515
Parent Process
Parent PID : 515
Child PID : 517
Parent is waiting...
Child Process
Child PID : 517
Parent PID : 515
Child is running...
k_anusri@DESKTOP-TM20BAQ:~$ ps -o pid,ppid,state,cmd -p <CHILD_PID>
-bash: syntax error near unexpected token `newline'
```

### Result

The child process was successfully created using `fork()`. The PID and PPID of the parent and child processes were displayed, and the process state was observed using the `ps` command.